

//Singly linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* Definition of node */
```

```
struct node {  
    int data;  
    struct node *next;  
};
```

```
struct node *head = NULL;
```

```
/* Function Prototypes */
```

```
void createe();
```

```
void insert5AA();
```

```
void deletee();
```

```
void traverse();
```

```
/* Main Function */
```

```
int main() {
```

```
    int choice;
```

```
    while (1) {
```

```
        printf("\n--- Singly Linked List Menu ---\n");
```

```
        printf("1. Create\n");
```

```
        printf("2. Insert5AA\n");
```

```
        printf("3. Delete\n");
```

```
        printf("4. Traverse\n");
```

```
        printf("5. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1: create();
```

```
                break;
```

```
            case 2: insert();
```

```
                break;
```

```
            case 3: deletee();
```

```
                break;
```

```
            case 4: traverse();
```

```

        break;
    case 5: exit(0);
    default: printf("Invalid choice\n");
    }
}
return 0;
}

```

/* Create Linked List */

```

void create() {
    int n, i, value;
    struct node *temp, *newnode;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &value);

        newnode->data = value;
        newnode->next = NULL;

        if (head == NULL) {
            head = newnode;
        } else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newnode;
        }
    }
}

```

/* Insert at End */

```

void insert() {
    int value;
    struct node *newnode, *temp;

```

```

newnode = (struct node *)malloc(sizeof(struct node));
printf("Enter value to insert: ");
scanf("%d", &value);

newnode->data = value;
newnode->next = NULL;

if (head == NULL) {
    head = newnode;
} else {
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newnode;
}
}

/* Delete a Node */
void deletee() {
    int value;
    struct node *temp, *prev;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &value);

    temp = head;

    if (temp->data == value) {
        head = temp->next;
        free(temp);
        printf("Node deleted\n");
        return;
    }

    while (temp != NULL && temp->data != value) {

```

```

    prev = temp;
    temp = temp->next;
}

if (temp == NULL) {
    printf("Value not found\n");
} else {
    prev->next = temp->next;
    free(temp);
    printf("Node deleted\n");
}
}

/* Traverse the List */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");}

```

--- Singly Linked List Menu ---

1. Create
2. Insert5AA
3. Delete
4. Traverse
5. Exit

Enter your choice: 1

Enter number of nodes: 3

Enter data: 45

Enter data: 65

Enter data: 75

--- Singly Linked List Menu ---

1. Create
2. Insert5AA
3. Delete
4. Traverse
5. Exit

Enter your choice: 2

Enter value to insert: 66

--- Singly Linked List Menu ---

1. Create
2. Insert5AA
3. Delete
4. Traverse
5. Exit

Enter your choice: 3

Enter value to delete: 45

Node deleted

```
--- Singly Linked List Menu ---
1. Create
2. Insert5AA
3. Delete
4. Traverse
5. Exit
Enter your choice: 4
Linked List: 65 -> 75 -> 66 -> NULL
```

```
--- Singly Linked List Menu ---
1. Create
2. Insert5AA
3. Delete
4. Traverse
5. Exit
Enter your choice: 5
```

```
-----
(program exited with code: 0)
Press return to continue
```

//Doubly linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* Definition of node */
```

```
struct node {  
    int data;  
    struct node *prev;  
    struct node *next;  
};
```

```
struct node *head = NULL;
```

```
/* Function Prototypes */
```

```
void create();
```

```
void insert();
```

```
void deletee();
```

```
void traverse();
```

```
/* Main Function */
```

```
int main() {
```

```
    int choice;
```

```
    while (1) {
```

```
        printf("\n--- Doubly Linked List Menu ---\n");
```

```
        printf("1. Create\n");
```

```
        printf("2. Insert\n");
```

```
        printf("3. Deletee\n");
```

```
        printf("4. Traverse\n");
```

```
        printf("5. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1: create();
```

```
                break;
```

```
            case 2: insert();
```

```
                break;
```

```
            case 3: deletee();
```

```
                break;
```

```
            case 4: traverse();
```

```
                break;
```

```
            case 5: exit(0);
```

```
            default: printf("Invalid choice\n");
```

```

    }
}
return 0;
}

```

/* Create Doubly Linked List */

```

void create() {
    int n, i, value;
    struct node *temp, *newnode;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &value);

        newnode->data = value;
        newnode->prev = NULL;
        newnode->next = NULL;

        if (head == NULL) {
            head = newnode;
        } else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newnode;
            newnode->prev = temp;
        }
    }
}

```

/* Insert at End */

```

void insert() {
    int value;
    struct node *newnode, *temp;

    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &value);

```



```
newnode->data = value;
newnode->prev = NULL;
newnode->next = NULL;
```

```
if (head == NULL) {
    head = newnode;
} else {
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newnode;
    newnode->prev = temp;
}
}
```

/* Delete a Node by Value */

```
void deletee() {
    int value;
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
```

```
    printf("Enter value to delete: ");
    scanf("%d", &value);
```

```
    temp = head;
```

/* If first node is to be deleted */

```
    if (temp->data == value) {
        head = temp->next;
        if (head != NULL)
            head->prev = NULL;
        free(temp);
        printf("Node deleted\n");
        return;
    }
```

```
    while (temp != NULL && temp->data != value)
        temp = temp->next;
```

```

if (temp == NULL) {
    printf("Value not found\n");
} else {
    temp->prev->next = temp->next;
    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    free(temp);
    printf("Node deleted\n");
}
}

```

/* Traverse Forward and Backward */

```

void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Forward Traversal: ");
    temp = head;
    while (temp->next != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("%d\n", temp->data);

    printf("Backward Traversal: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}

```

```
--- Doubly Linked List Menu ---
1. Create
2. Insert
3. Deletee
4. Traverse
5. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 53
Enter data: 76
Enter data: 66

--- Doubly Linked List Menu ---
1. Create
2. Insert
3. Deletee
4. Traverse
5. Exit
Enter your choice: 2
Enter value to insert: 3

--- Doubly Linked List Menu ---
1. Create
2. Insert
3. Deletee
4. Traverse
5. Exit
Enter your choice: 3
Enter value to delete: 3
Node deleted

--- Doubly Linked List Menu ---
1. Create
2. Insert
3. Deletee
4. Traverse
5. Exit
Enter your choice: 4
Forward Traversal: 53 <-> 76 <-> 66
Backward Traversal: 66 <-> 76 <-> 53 <-> NULL

--- Doubly Linked List Menu ---
1. Create
2. Insert
3. Deletee
4. Traverse
5. Exit
Enter your choice: 5

-----
(program exited with code: 0)
```

//Circular linked list

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

/* Function declarations */
void create();
void insert();
void delete();
void traverse();

/* Main function */
int main() {
    int choice;

    while (1) {
        printf("\n--- Circular Linked List Menu ---\n");
        printf("1. Create\n");
        printf("2. Insert\n");
        printf("3. Delete\n");
        printf("4. Traverse\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: create(); break;
            case 2: insert(); break;
            case 3: delete(); break;
            case 4: traverse(); break;
            case 5: exit(0);
            default: printf("Invalid choice\n");
        }
    }
    return 0;
}
```

```

/* Create circular linked list */
void create() {
    int n, i, val;
    struct node *newnode, *temp;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &val);

        newnode->data = val;

        if (head == NULL) {
            head = newnode;
            newnode->next = head;
        } else {
            temp = head;
            while (temp->next != head)
                temp = temp->next;

            temp->next = newnode;
            newnode->next = head;
        }
    }
}

```

```

/* Insert at end */
void insert() {
    int val;
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &val);

    newnode->data = val;

    if (head == NULL) {
        head = newnode;
        newnode->next = head;
    } else {

```

```

    temp = head;
    while (temp->next != head)
        temp = temp->next;

    temp->next = newnode;
    newnode->next = head;
}
}

```

/* Delete node by value */

```

void delete() {
    int val;
    struct node *temp, *prev;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &val);

    temp = head;
    prev = NULL;

    /* If head node is to be deleted */
    if (temp->data == val) {
        if (temp->next == head) {
            free(temp);
            head = NULL;
            printf("Node deleted\n");
            return;
        }
        prev = head;
        while (prev->next != head)
            prev = prev->next;

        prev->next = temp->next;
        head = temp->next;
        free(temp);
        printf("Node deleted\n");
        return;
    }
}

```

```

prev = temp;
temp = temp->next;

while (temp != head && temp->data != val) {
    prev = temp;
    temp = temp->next;
}

if (temp == head) {
    printf("Value not found\n");
} else {
    prev->next = temp->next;
    free(temp);
    printf("Node deleted\n");
}
}

/* Traverse circular linked list */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Circular Linked List: ");
    temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(back to head)\n");
}

```

```
--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 23
Enter data: 45
Enter data: 65

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 2
Enter value to insert: 44

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 3
Enter value to delete: 23
Node deleted

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 4
Circular Linked List: 45 -> 65 -> 44 -> (back to head)

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 5

-----
(program exited with code: 0)
Press return to continue
```


//Stack using Arrays

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define MAX 5
```

```
int stack[MAX];
int top = -1;
```

```
/* Function Prototypes */
```

```
void push();
void pop();
void peek();
void traverse();
```

```
/* Main Function */
```

```
int main()
{
    int choice;

    while(1)
    {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Traverse\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d",&choice);

        switch(choice)
        {
            case 1: push();
                    break;

            case 2: pop();
```

```
        break;

    case 3: peek();
        break;

    case 4: traverse();
        break;

    case 5: exit(0);

    default: printf("Invalid choice\n");
    }
}

return 0;
}

/* Push Operation */

void push()
{
    int value;

    if(top == MAX-1)
    {
        printf("Stack Overflow \n");
        return;
    }

    printf("Enter value to push: ");
    scanf("%d",&value);

    top = top + 1;
    stack[top] = value;

    printf("Element inserted\n");
}

/* Pop Operation */
```

```
void pop()
{
    if(top == -1)
    {
        printf("Stack Underflow\n");
        return;
    }

    printf("Deleted element is %d\n",stack[top]);
    top = top - 1;
}
```

/* Peek Operation */

```
void peek()
{
    if(top == -1)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Top element is %d\n",stack[top]);
}
```

/* Traverse Stack */

```
void traverse()
{
    int i;

    if(top == -1)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements are:\n");

    for(i = top; i >= 0; i--)
```

```
    {  
        printf("%d\n",stack[i]);  
        temp = temp->next;  
    }  
}
```

```
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 3  
Top element is 5  
  
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 4  
Stack elements are:  
5  
  
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 2  
Deleted element is 5  
  
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 5  
  
-----  
(program exited with code: 0)  
Press return to continue
```

//Stack using Linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* Definition of Node */
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *top = NULL;
```

```
/* Function Prototypes */
```

```
void push();
```

```
void pop();
```

```
void peek();
```

```
void traverse();
```

```
/* Main Function */
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while(1)
```

```
    {
```

```
        printf("\n--- Stack Menu ---\n");
```

```
        printf("1. Push\n");
```

```
        printf("2. Pop\n");
```

```
        printf("3. Peek\n");
```

```
        printf("4. Traverse\n");
```

```
        printf("5. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d",&choice);
```

```
        switch(choice)
```

```
        {
```

```
            case 1: push();
```

```
                break;
```

```
            case 2: pop();
```

```

        break;

    case 3: peek();
        break;

    case 4: traverse();
        break;

    case 5: exit(0);

    default: printf("Invalid choice\n");
}
}

return 0;
}

/* Push Operation */

void push()
{
    int value;
    struct node *newnode;

    newnode = (struct node*)malloc(sizeof(struct node));

    printf("Enter value to push: ");
    scanf("%d",&value);

    newnode->data = value;
    newnode->next = top;
    top = newnode;

    printf("Element inserted\n");
}

/* Pop Operation */

void pop()
{
    struct node *temp;

    if(top == NULL)
    {

```

```

        printf("Stack Underflow\n");
        return;
    }

    temp = top;
    printf("Deleted element is %d\n", top->data);

    top = top->next;
    free(temp);
}

```

/* Peek Operation */

```

void peek()
{
    if(top == NULL)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Top element is %d\n", top->data);
}

```

/* Traverse Stack */

```

void traverse()
{
    struct node *temp;

    if(top == NULL)
    {
        printf("Stack is empty\n");
        return;
    }

    temp = top;

    printf("Stack elements are:\n");

    while(temp != NULL)
    {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

```

```
}  
}
```

```
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 1  
Enter value to push: 4  
Element inserted  
  
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 3  
Top element is 4  
  
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 4  
Stack elements are:  
4  
  
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 2  
Deleted element is 4  
  
--- Stack Menu ---  
1. Push  
2. Pop  
3. Peek  
4. Traverse  
5. Exit  
Enter your choice: 5  
  
-----  
(program exited with code: 0)  
Press return to continue
```


//Queue using Arrays

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 5
```

```
int queue[MAX];
```

```
int front = -1, rear = -1;
```

```
/* Function declarations */
```

```
void enqueue();
```

```
void dequeue();
```

```
void peek();
```

```
void display();
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while(1)
```

```
    {
```

```
        printf("\n--- Queue Menu ---\n");
```

```
        printf("1. Enqueue(insert)\n");
```

```
        printf("2. Dequeue(delete)\n");
```

```
        printf("3. Peek\n");
```

```
        printf("4. Display\n");
```

```
        printf("5. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch(choice)
```

```
        {
```

```
            case 1: enqueue(); break;
```

```
            case 2: dequeue(); break;
```

```
            case 3: peek(); break;
```

```
            case 4: display(); break;
```

```
            case 5: exit(0);
```

```
            default: printf("Invalid choice\n");
```

```
        }
```

```
    }
```

```
}
```

```
/* Enqueue operation */
```

```
void enqueue()
```

```

{
    int item;

    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter element to insert: ");
    scanf("%d", &item);

    if(front == -1)
        front = 0;

    rear++;
    queue[rear] = item;

    printf("Element inserted successfully\n");
}

/* Dequeue operation */
void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("Deleted element: %d\n", queue[front]);
    front++;

    if(front > rear)
    {
        front = rear = -1;
    }
}

/* Peek operation */
void peek()
{
    if(front == -1)
    {

```

```
        printf("Queue is empty\n");
        return;
    }

    printf("Front element: %d\n", queue[front]);
}

/* Display operation */
void display()
{
    int i;

    if(front == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements:\n");
    for(i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }
    printf("\n");
}
```

```
--- Queue Menu ---
1. Enqueue(insert)
2. Dequeue(delete)
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to insert: 4
Element inserted successfully
```

```
--- Queue Menu ---
1. Enqueue(insert)
2. Dequeue(delete)
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 4
```

```
--- Queue Menu ---
1. Enqueue(insert)
2. Dequeue(delete)
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
4
```

```
--- Queue Menu ---
1. Enqueue(insert)
2. Dequeue(delete)
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 4
```

```
--- Queue Menu ---
1. Enqueue(insert)
2. Dequeue(delete)
3. Peek
4. Display
5. Exit
Enter your choice: 5
```

```
-----
(program exited with code: 0)
Press return to continue
```

//Queue using Linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *front = NULL, *rear = NULL;
```

```
/* Function declarations */
```

```
void enqueue();
```

```
void dequeue();
```

```
void peek();
```

```
void display();
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    while(1)
```

```
    {
```

```
        printf("\n--- Queue Using Linked List ---\n");
```

```
        printf("1. Enqueue\n");
```

```
        printf("2. Dequeue\n");
```

```
        printf("3. Peek\n");
```

```
        printf("4. Display\n");
```

```
        printf("5. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch(choice)
```

```
        {
```

```
            case 1: enqueue(); break;
```

```
            case 2: dequeue(); break;
```

```
            case 3: peek(); break;
```

```
            case 4: display(); break;
```

```
            case 5: exit(0);
```

```
            default: printf("Invalid choice\n");
```

```
        }
```

```
    }
```

```
}
```

```

/* Enqueue operation */
void enqueue()
{
    struct node *ptr;
    int value;

    ptr = (struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter element to insert: ");
    scanf("%d", &value);

    ptr->data = value;
    ptr->next = NULL;

    if(rear == NULL)
    {
        front = rear = ptr;
    }
    else
    {
        rear->next = ptr;
        rear = ptr;
    }

    printf("Element inserted successfully\n");
}

```

```

/* Dequeue operation */
void dequeue()
{
    struct node *temp;

    if(front == NULL)
    {
        printf("Queue Underflow\n");
        return;
    }

```

```

        temp = front;
        printf("Deleted element: %d\n", temp->data);
        front = front->next;

        if(front == NULL)
            rear = NULL;

        free(temp);
    }

/* Peek operation */
void peek()
{
    if(front == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Front element: %d\n", front->data);
}

/* Display operation */
void display()
{
    struct node *temp;

    if(front == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    temp = front;
    printf("Queue elements: ");
    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

--- Queue Using Linked List ---

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter element to insert: 5

Element inserted successfully

--- Queue Using Linked List ---

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 3

Front element: 5

--- Queue Using Linked List ---

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 4

Queue elements: 5

--- Queue Using Linked List ---

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 2

Deleted element: 5

--- Queue Using Linked List ---

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 5

(program exited with code: 0)

Press return to continue

//Radix Sort

```
#include <stdio.h>
#include <stdlib.h>
// Function to find maximum element
int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}
// CountingSort based on digit
void countingSort(int arr[], int n, int place) {
    int *output = (int *)malloc(n * sizeof(int));
    int count[10] = {0};
    // Count occurrences
    for (int i = 0; i < n; i++) {
        int digit = (arr[i] / place) % 10;
        count[digit]++;
    }
    // Cumulative count
    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }
    // Build Output array
    for (int i = n - 1; i >= 0; i--) {
        int digit = (arr[i] / place) % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }
    // Copy back to Original array
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
    free(output);
}
// RadixSort
void radixSort(int arr[], int n) {
    int max = getMax(arr, n);
    for (int place = 1; max / place > 0; place *= 10) {
        countingSort(arr, n, place);
    }
}
```

```

// Print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

//Main function
int main() {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 1;

    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Original array:\n");
    printArray(arr, n);
    radixSort(arr, n);
    printf("Sorted array:\n");
    printArray(arr, n);
    free(arr); //Free allocated memory
    return 0;
}

```

Enter number of elements: 5

Enter 5 elements:

76

58

43

23

95

Original array:

76 58 43 23 95

Sorted array:

23 43 58 76 95

(program exited with code: 0)

Press return to continue

//Heap Sort

```
#include <stdio.h>
```

```
// Function to heapify a subtree rooted at index i
```

```
void heapify(int arr[], int n, int i) {
```

```
    int largest = i;    // Assume root is largest
```

```
    int left = 2 * i + 1; // Left child
```

```
    int right = 2 * i + 2; // Right child
```

```
    // If left child is larger
```

```
    if (left < n && arr[left] > arr[largest])
```

```
        largest = left;
```

```
    // If right child is larger
```

```
    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;
```

```
    // If largest is not root
```

```
    if (largest != i) {
```

```
        int temp = arr[i];
```

```
        arr[i] = arr[largest];
```

```
        arr[largest] = temp;
```

```
        // Recursively heapify affected subtree
```

```
        heapify(arr, n, largest);
```

```
    }
```

```
}
```

```
// Heap Sort function
```

```
void heapSort(int arr[], int n) {
```

```
    // Build max heap
```

```
    for (int i = n / 2 - 1; i >= 0; i--)
```

```
        heapify(arr, n, i);
```

```
    // Extract elements one by one
```

```
    for (int i = n - 1; i > 0; i--) {
```

```
        // Swap root with last element
```

```
        int temp = arr[0];
```

```
        arr[0] = arr[i];
```

```
        arr[i] = temp;
```

```

        // Heapify reduced heap
        heapify(arr, i, 0);
    }
}

// Print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

// Driver code
int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

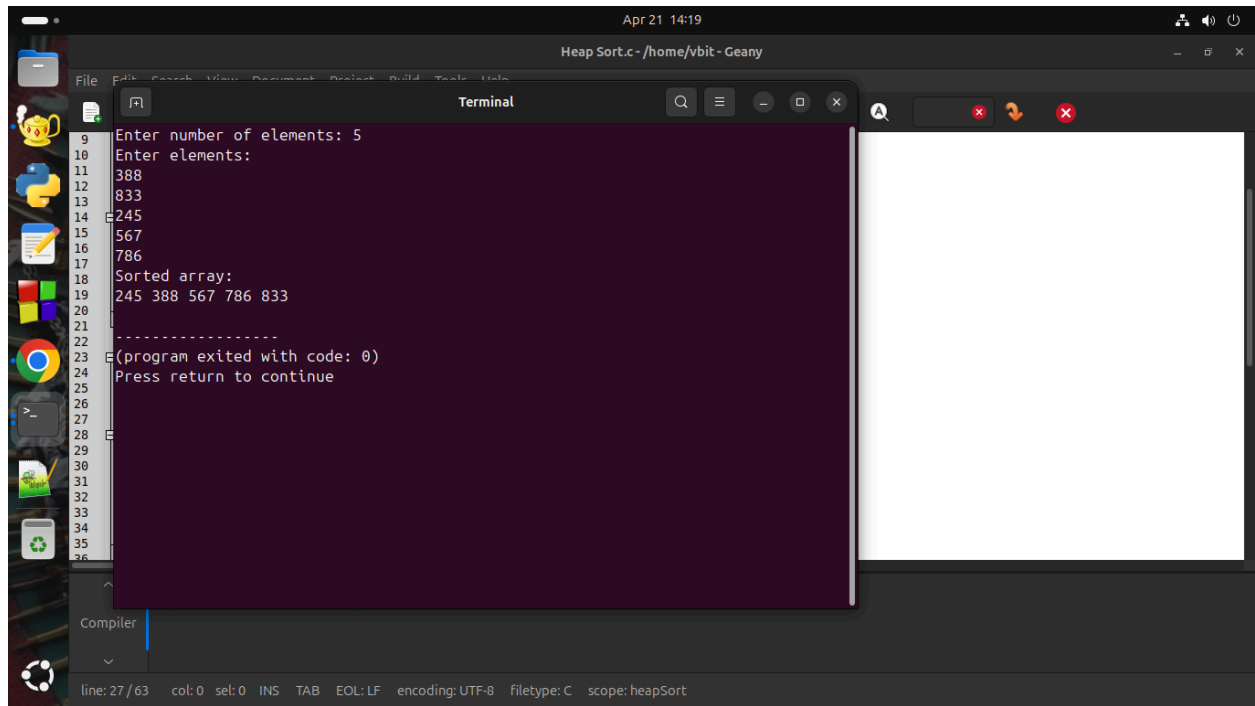
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array:\n");
    printArray(arr, n);

    return 0;
}

```



Apr 21 14:19

Heap Sort.c - /home/vbit - Geany

File Edit Search View Document Project Build Tools Help

Terminal

```
9 Enter number of elements: 5
10 Enter elements:
11 388
12 833
13 245
14 567
15 786
16 Sorted array:
17 245 388 567 786 833
18
19 -----
20
21
22
23 (program exited with code: 0)
24 Press return to continue
25
26
27
28
29
30
31
32
33
34
35
36
```

Compiler

line: 27 / 63 col: 0 sel: 0 INS TAB EOL: LF encoding: UTF-8 filetype: C scope: heapSort

//Shell Sort

```
#include <stdio.h>
```

```
// Function to perform Shell Sort
```

```
void shellSort(int arr[], int n) {  
    int gap, i, j, temp;
```

```
    // Start with a large gap, then reduce the gap  
    for (gap = n / 2; gap > 0; gap /= 2) {
```

```
        // Perform insertion sort for this gap  
        for (i = gap; i < n; i++) {  
            temp = arr[i];
```

```
            // Shift earlier gap-sorted elements up  
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {  
                arr[j] = arr[j - gap];  
            }
```

```
            arr[j] = temp;  
        }  
    }  
}
```

```
// Function to print array
```

```
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
}
```

```
// Main function
```

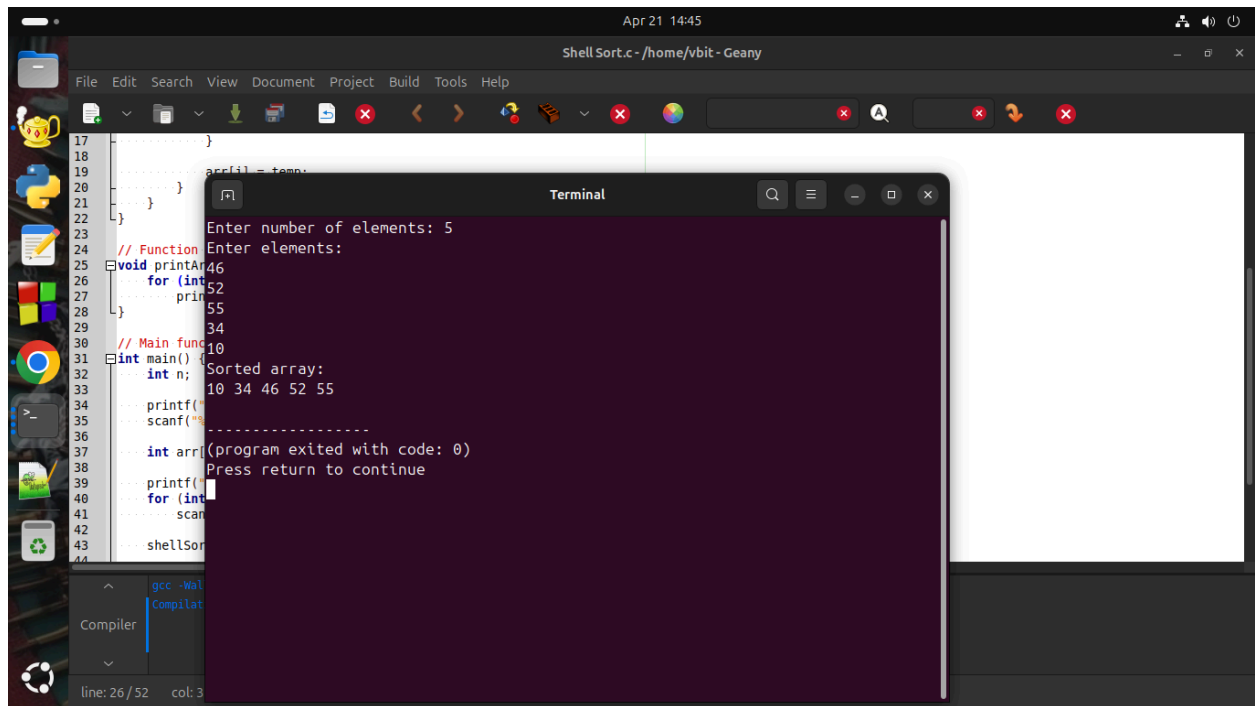
```
int main() {  
    int n;  
  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    int arr[n];  
  
    printf("Enter elements:\n");
```

```
for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

shellSort(arr, n);

printf("Sorted array:\n");
printArray(arr, n);

return 0;
}
```



The screenshot shows the Geany IDE with a C program for Shell Sort. The code is as follows:

```
17 }
18 ..... arrfil = fopen
19 ..... }
20 ..... }
21 ..... }
22 ..... }
23 ..... }
24 ..... }
25 // Function
26 void printA
27 ..... for (int
28 ..... print
29 ..... }
30 ..... }
31 // Main func
32 int main()
33 ..... int n;
34 ..... printf(
35 ..... scanf(
36 ..... }
37 ..... int arr[
38 ..... }
39 ..... printf(
40 ..... for (int
41 ..... scanf
42 ..... }
43 ..... shellSor
44 }
```

The terminal output shows the program's execution:

```
Enter number of elements: 5
Enter elements:
10 34 46 52 55
Sorted array:
10 34 46 52 55
(program exited with code: 0)
Press return to continue
```


//Tree Sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define structure for BST node
```

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

```
// Create a new node
```

```
struct Node* createNode(int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = value;  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

```
// Insert into BST
```

```
struct Node* insert(struct Node* root, int value) {  
    if (root == NULL)  
        return createNode(value);  
  
    if (value < root->data)  
        root->left = insert(root->left, value);  
    else  
        root->right = insert(root->right, value);  
  
    return root;  
}
```

```
// Inorder traversal (sorted order)
```

```
void inorder(struct Node* root) {  
    if (root != NULL) {  
        inorder(root->left);  
        printf("%d ", root->data);  
        inorder(root->right);  
    }  
}
```

```

// Main function
int main() {
    int n, i, value;
    struct Node* root = NULL;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Sorted elements:\n");
    inorder(root);

    return 0;
}

```

```

Tree Sort.c - /home/vbit - Geany
File Edit Search View Document Project Build Tools Help
34 if (root != NULL) {
35     inorder(root->left);
36     printf("%d ", root->data);
37     inorder(root->right);
38 }
39 }
40
41 // Main function
42 int main() {
43     int n, i;
44     struct Node* root = NULL;
45     printf("Enter number of elements: ");
46     scanf("%d", &n);
47     printf("Enter elements:\n");
48     for (i = 0; i < n; i++) {
49         int value;
50         scanf("%d", &value);
51         root = insert(root, value);
52     }
53     printf("Sorted elements:\n");
54     inorder(root);
55     return 0;
56 }
57
58
59
60
61

```

Terminal

```

Enter number of elements: 5
Enter elements:
23 34 43 56 76
Sorted elements:
23 34 43 56 76
(program exited with code: 0)
Press return to continue

```

Compiler: gcc -Wall -g -O2 -c

line: 60 / 61 col: 0

//Tree Traversal (Recursive and Non Recursive)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure for tree node
```

```
struct Node {  
    int data;  
    struct Node *left, *right;  
};
```

```
// Create new node
```

```
struct Node* createNode(int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

```
// ♦ RECURSIVE TRAVERSALS
```

```
void inorderRecursive(struct Node* root) {  
    if (root != NULL) {  
        inorderRecursive(root->left);  
        printf("%d ", root->data);  
        inorderRecursive(root->right);  
    }  
}
```

```
void preorderRecursive(struct Node* root) {  
    if (root != NULL) {  
        printf("%d ", root->data);  
        preorderRecursive(root->left);  
        preorderRecursive(root->right);  
    }  
}
```

```
void postorderRecursive(struct Node* root) {  
    if (root != NULL) {  
        postorderRecursive(root->left);  
        postorderRecursive(root->right);  
        printf("%d ", root->data);  
    }  
}
```

```
// ♦ NON-RECURSIVE TRAVERSALS (Using Stack)
```

```

#define MAX 100

struct Node* stack[MAX];
int top = -1;

void push(struct Node* node) {
    stack[++top] = node;
}

struct Node* pop() {
    return stack[top--];
}

int isEmpty() {
    return top == -1;
}

// Non-Recursive Inorder
void inorderNonRecursive(struct Node* root) {
    struct Node* current = root;

    while (current != NULL || !isEmpty()) {
        while (current != NULL) {
            push(current);
            current = current->left;
        }

        current = pop();
        printf("%d ", current->data);
        current = current->right;
    }
}

// Non-Recursive Preorder
void preorderNonRecursive(struct Node* root) {
    if (root == NULL) return;

    push(root);

    while (!isEmpty()) {
        struct Node* temp = pop();
        printf("%d ", temp->data);

        if (temp->right)

```

```

        push(temp->right);
    if (temp->left)
        push(temp->left);
    }
}

```

```

// Non-Recursive Postorder (using two stacks)
void postorderNonRecursive(struct Node* root) {
    if (root == NULL) return;

```

```

    struct Node* stack1[MAX];
    struct Node* stack2[MAX];
    int top1 = -1, top2 = -1;

```

```

    stack1[++top1] = root;

```

```

    while (top1 != -1) {
        struct Node* temp = stack1[top1--];
        stack2[++top2] = temp;

```

```

        if (temp->left)
            stack1[++top1] = temp->left;
        if (temp->right)
            stack1[++top1] = temp->right;
    }

```

```

    while (top2 != -1) {
        printf("%d ", stack2[top2--]->data);
    }
}

```

```

// ♦ MAIN FUNCTION

```

```

int main()

```

```

{
    // Creating sample tree
    /*
        1
       /\
      2 3
     /\
    4 5
    */

```

```

    struct Node* root = createNode(1);
    root->left = createNode(2);

```

```
root->right = createNode(3);
root->left->left = createNode(4);
root->left->right = createNode(5);

printf("Recursive Traversals:\n");

printf("Inorder: ");
inorderRecursive(root);

printf("\nPreorder: ");
preorderRecursive(root);

printf("\nPostorder: ");
postorderRecursive(root);

printf("\n\nNon-Recursive Traversals:\n");

printf("Inorder: ");
inorderNonRecursive(root);

printf("\nPreorder: ");
preorderNonRecursive(root);

printf("\nPostorder: ");
postorderNonRecursive(root);

return 0;
}
```

Apr 21 15:00



Terminal



Recursive Traversals:

Inorder: 4 2 5 1 3

Preorder: 1 2 4 5 3

Postorder: 4 5 2 3 1

Non-Recursive Traversals:

Inorder: 4 2 5 1 3

Preorder: 1 2 4 5 3

Postorder: 4 5 2 3 1

(program exited with code: 0)

Press return to continue



//Binary search, B Trees, B+Trees, AVL Trees

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
////////////////////////////////////
```

```
// ♦ BST & AVL COMMON STRUCTURE
```

```
////////////////////////////////////
```

```
struct Node {  
    int data;  
    struct Node *left, *right;  
    int height; // used only for AVL  
};
```

```
int max(int a, int b) { return (a > b) ? a : b; }
```

```
int height(struct Node* n) {  
    return n ? n->height : 0;  
}
```

```
struct Node* createNode(int data) {  
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));  
    n->data = data;  
    n->left = n->right = NULL;  
    n->height = 1;  
    return n;  
}
```

```
////////////////////////////////////
```

```
// ♦ BST FUNCTIONS
```

```
////////////////////////////////////
```

```
struct Node* insertBST(struct Node* root, int data) {  
    if (root == NULL) return createNode(data);  
  
    if (data < root->data)  
        root->left = insertBST(root->left, data);  
    else  
        root->right = insertBST(root->right, data);
```



```

    return root;
}

////////////////////////////////////
// ♦ AVL FUNCTIONS
////////////////////////////////////

int getBalance(struct Node* n) {
    return n ? height(n->left) - height(n->right) : 0;
}

struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

struct Node* insertAVL(struct Node* node, int key) {
    if (node == NULL) return createNode(key);

```

```

if (key < node->data)
    node->left = insertAVL(node->left, key);
else if (key > node->data)
    node->right = insertAVL(node->right, key);
else
    return node;

node->height = 1 + max(height(node->left), height(node->right));

int balance = getBalance(node);

if (balance > 1 && key < node->left->data)
    return rightRotate(node);

if (balance < -1 && key > node->right->data)
    return leftRotate(node);

if (balance > 1 && key > node->left->data) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
}

if (balance < -1 && key < node->right->data) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
}

return node;
}

```

```

////////////////////////////////////
// ♦ SIMPLE B-TREE (DEMO)
////////////////////////////////////

```

```

#define MAX 3

```

```

struct BTree {
    int val[MAX];
    int count;
};

```

```

void insertBTree(struct BTree* t, int val) {
    if (t->count < MAX) {
        t->val[t->count++] = val;
    } else {
        printf("B-Tree node full (demo only)\n");
    }
}

```

```

void displayBTree(struct BTree* t) {
    for (int i = 0; i < t->count; i++)
        printf("%d ", t->val[i]);
}

```

```

////////////////////////////////////
// ♦ SIMPLE B+ TREE (DEMO)
////////////////////////////////////

```

```

struct BPlus {
    int keys[MAX];
    int count;
};

```

```

void insertBPlus(struct BPlus* t, int val) {
    if (t->count < MAX) {
        t->keys[t->count++] = val;
    } else {
        printf("B+ Tree node full (demo only)\n");
    }
}

```

```

void displayBPlus(struct BPlus* t) {
    for (int i = 0; i < t->count; i++)
        printf("%d ", t->keys[i]);
}

```

```

////////////////////////////////////
// ♦ COMMON TRAVERSAL
////////////////////////////////////

```

```

void inorder(struct Node* root) {
    if (root) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

```

```

////////////////////////////////////
// ♦ MAIN MENU
////////////////////////////////////

```

```

int main() {
    int choice, val, n;

    struct Node* bstRoot = NULL;
    struct Node* avlRoot = NULL;
    struct BTree bt = {.count = 0};
    struct BPlus bpt = {.count = 0};

    while (1) {
        printf("\n===== TREE MENU =====\n");
        printf("1. Binary Search Tree\n");
        printf("2. AVL Tree\n");
        printf("3. B-Tree (Demo)\n");
        printf("4. B+ Tree (Demo)\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 5) break;

        printf("Enter number of elements: ");
        scanf("%d", &n);

        switch (choice) {

        case 1:
            bstRoot = NULL;
            printf("Enter elements:\n");

```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &val);  
    bstRoot = insertBST(bstRoot, val);  
}  
printf("BST Inorder: ");  
inorder(bstRoot);  
break;
```

case 2:

```
avlRoot = NULL;  
printf("Enter elements:\n");  
for (int i = 0; i < n; i++) {  
    scanf("%d", &val);  
    avlRoot = insertAVL(avlRoot, val);  
}  
printf("AVL Inorder: ");  
inorder(avlRoot);  
break;
```

case 3:

```
bt.count = 0;  
printf("Enter elements:\n");  
for (int i = 0; i < n; i++) {  
    scanf("%d", &val);  
    insertBTree(&bt, val);  
}  
printf("B-Tree elements: ");  
displayBTree(&bt);  
break;
```

case 4:

```
bpt.count = 0;  
printf("Enter elements:\n");  
for (int i = 0; i < n; i++) {  
    scanf("%d", &val);  
    insertBPlus(&bpt, val);  
}  
printf("B+ Tree elements: ");  
displayBPlus(&bpt);  
break;
```

Apr 21 15:18

Terminal

```
==== TREE MENU ====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Deno)
4. B+ Tree (Deno)
5. Exit
Enter choice: 1
Enter number of elements: 3
Enter elements:
45
76
34
BST Inorder: 34 45 76
==== TREE MENU ====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Deno)
4. B+ Tree (Deno)
5. Exit
Enter choice: 2
Enter number of elements: 3
Enter elements:
65
56
45
AVL Inorder: 45 56 65
==== TREE MENU ====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Deno)
4. B+ Tree (Deno)
5. Exit
Enter choice: 3
Enter number of elements: 3
Enter elements:
56
34
22
B-Tree elements: 56 34 22
==== TREE MENU ====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Deno)
4. B+ Tree (Deno)
5. Exit
Enter choice: 4
Enter number of elements: 3
Enter elements:
86
65
47
B+ Tree elements: 86 65 47
==== TREE MENU ====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Deno)
4. B+ Tree (Deno)
5. Exit
Enter choice: 5
```